EduPay Africa – Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

The purpose of EduPay Africa is to provide a **digital platform for education finance management**. The system allows parents, students, and institutions to manage, track,

and report fee payments securely. It replaces manual, paper-based processes with a centralized, verifiable, and transparent platform.

1.2 Scope

- **Users:** Parents, students, school/college administrators.
- **Platform:** Web-only (desktop and mobile browsers).
- **Key capabilities:**
  - Role-based dashboards
  - Multi-institution parent support
  - Historical payment tracking
  - Fee management for institutions
  - Downloadable statements
  - Secure authentication and data isolation
  - School demo request and approval workflow
  - Mass student upload per school with template
  - M-Pesa payment integration via Daraja API
- **Local development:** XAMPP stack for PHP, MySQL, and Apache hosting

1.3 Definitions, Acronyms, and Abbreviations

TERM	DEFINITION
MVP	Minimum Viable Product
LAMP	Linux, Apache, MySQL, PHP
XAMPP	Cross-platform Apache, MySQL, PHP, Perl local development stack
CRUD	Create, Read, Update, Delete
TCPDF/FPDF	PHP libraries for PDF generation
Daraja API	Safaricom M-Pesa API for payment integration

1.4 References

- MySQL 8 Documentation
- PHP 8+ Documentation
- Safaricom Daraja API Documentation
- XAMPP Documentation

2. Overall Description

2.1 Product Perspective

EduPay Africa is a **standalone web application** for digital fee management. It uses **MySQL** for data storage, **PHP** for backend logic, and **HTML/CSS/JS** for frontend UI. For local development, **XAMPP** will be used to host Apache, PHP, and MySQL on Windows, macOS, or Linux.

High-level architecture:

Architecture Diagram

Architecture Diagram

2.2 Product Functions

1. Parent/Student Dashboard:
    - View linked students
    - See fee breakdowns, payment history, outstanding balances
    - Download institution-branded statements
2. Institution Admin Panel:
    - Manage students, parents, fee structures
    - Approve/deny parent-student link requests
    - Generate reports
3. Authentication:
    - Secure login via PHP-based authentication with password hashing (bcrypt)
    - Role-based access enforcement
4. Multi-Institution Support:
    - Parents with children in multiple institutions can view all linked students
5. School Demo Request:
    - Schools can submit demo requests with fields: school name, location, contact email, phone, principal name, and additional info
    - Requests stored in a demo request table
    - Superadmin approves request → school added to system
    - Confirmation email sent with link to set password and complete remaining school details
6. Mass Student Upload:
    - Admins can upload multiple students per school using downloadable CSV template
    - System validates uploaded data
7. Payment Integration:
    - M-Pesa payments handled via Daraja API for fee collections

2.3 User Classes and Characteristics

User Class	Role	Access
Parent	Parent/Guardian	View dashboard, download statements,

		multi-institution view
Student	Student	View personal dashboard, payment history
Institution Admin	Staff	Manage students, parents, fee structures, approve links, mass upload students
Superadmin	Platform Admin	Approve demo requests, create schools, configure payment integration

2.4 Operating Environment

- Web browser: Chrome, Firefox, Edge, Safari
- Server: Apache (via XAMPP) with PHP 8+
- Database: MySQL 8+ (InnoDB) hosted via XAMPP or production server
- Hosting: Local XAMPP for development, cloud or on-premise LAMP stack for production

2.5 Design and Implementation Constraints

- Web-only MVP; no mobile app at launch
- PHP-based authentication (no Firebase)
- Multi-institution parent support required from MVP
- Data privacy regulations (GDPR-like) must be followed
- Must support at least 5000 concurrent parent/student users at launch

2.6 Assumptions and Dependencies

- Users have internet access and a modern browser
- Institutions provide verified student and parent contact information
- Demo requests and manual school registrations supported
- Local development uses XAMPP stack
- Future digital payment expansion possible beyond M-Pesa

3. System Features and Requirements

3.1 Functional Requirements

1. Authentication - PHP-based login and registration for all users - Passwords hashed using bcrypt - Role-based access control: - Parents/Students: Read-only dashboard - Admins: Full CRUD access - Superadmin: Demo approval, school creation, and system management - Password reset via email link

2. Parent/Student Dashboard - Display all linked students and institutions - Show fee structures, balances, and payment history - Download PDF statements (per student)

3. Institution Admin Panel - CRUD operations for students, parents, fee structures - Approve/deny parent-student link requests - Mass upload students using downloadable CSV template - Generate reports (payment history, overdue balances) - Institution-level data isolation

4. Payment Tracking and Integration - Record payment entries (amount, date, method) - Auto-calculate outstanding balances - Prevent duplicate payment entries - Integrate M-Pesa payments using Daraja API

5. Multi-Institution Parent Support - Parent account can link to multiple institutions - Data isolation maintained per institution - Dashboard aggregates all linked student data

6. School Demo Request Workflow - Schools submit demo requests with required fields: school name, location, contact email, phone, principal name - Demo requests stored in database - Superadmin reviews and approves - Confirmation email sent with link to set password and fill remaining school details - Approved school automatically created in system

7. Manual School Registration - Superadmin can manually add schools without demo request - Email verification and account setup link generated

3.2 Non-Functional Requirements

1. Performance - Page load under 3 seconds - Support 5000 concurrent users at MVP scale - Database queries optimized with indexes on student_id, parent_id, institution_id

2. Security - HTTPS mandatory - Passwords hashed securely - Role-based access enforced at backend - SQL injection and XSS prevention - Institution-scoped queries

3. Reliability & Availability - 99.5% uptime - Daily database backups - Error logging

4. Maintainability - Modular PHP code (MVC pattern recommended) - Database normalization (3NF) - Clear documentation

5. Usability - Intuitive UI, responsive design - Minimal training required

3.3 Interface Requirements

- HTML5/CSS3 dashboards, JS interactivity
- Forms for student, parent, fee structure CRUD
- Downloadable PDF statement generation
- CSV template download for student mass upload

3.4 Data Requirements

- Tables: institutions, parents, students, fee_structures, payments, parent_student_link, demo_requests
- Foreign keys for institution_id, student_id, parent_id

- Unique constraints on email/phone
- Proper indexing for queries

3.5 Security Requirements

- Enforce password policies (min 8 chars, alphanumeric)
- Role-based access control
- HTTPS mandatory
- SQL injection and XSS prevention
- Audit logs for admin actions

3.6 System Evolution

- Expand payment methods beyond M-Pesa
- Mobile app support
- Analytics dashboards for institutions
- Notifications (SMS/email) for parents/students